

Practical 5:- MongoDB Integration and Schema Design with Mongoose

Objective:	To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema.
Problem Statement:	• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose. • Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now). • Replace the in-memory array from Practical 4 with real Mongoose model operations. • Test all CRUD operations against the live database using Postman. • Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.
Implementation	Task Card: <pre>import { FiClock, FiEdit2, FiTrash2 } from 'react-icons/fi'; import { formatDueDate } from '../utils/format.js'; import { cn, paperCard, gloss, paperCardLift, badge, badgeGreen, badgeYellow, badgeRed, badgeBlue, statusDot, btnIcon, btnIconDanger, notebookAccent, accentHigh, accentMedium, accentLow, } from '../styles/classes.js'; const STATUS_META = { pending: { label: 'Pending', badge: badgeYellow, dot: 'bg-status-yellow' }, in_progress: { label: 'In Progress', badge: badgeBlue, dot: 'bg-status-blue' }, completed: { label: 'Completed', badge: badgeGreen, dot: 'bg-status-green' }, }; const PRIORITY_META = { high: { label: 'High', badge: badgeRed, accent: accentHigh }, medium: { label: 'Medium', badge: badgeYellow, accent: accentMedium }, low: { label: 'Low', badge: badgeGreen, accent: accentLow }, }; const isOverdue = (task) => { if (task.status === 'completed' !task.dueDate) return false; const due = new Date(`\${task.dueDate}T00:00:00`); const today = new Date(); today.setHours(0, 0, 0, 0); return !Number.isNaN(due.getTime()) && due < today;</pre>

```

};

const TaskCard = ({ task, onEdit, onDelete }) => {
  const status = STATUS_META[task.status] || STATUS_META.pending;
  const priority = PRIORITY_META[task.priority] || PRIORITY_META.medium;
  const isCompleted = task.status === 'completed';
  const overdue = isOverdue(task);

  return (
    <article className={cn(paperCard, gloss, paperCardLight, 'flex h-full flex-col p-5')}>
      <div className={cn(notebookAccent, priority.accent)} aria-hidden="true" />

      <div className={cn('flex items-center justify-between gap-3')}>
        <div className={cn('flex flex-wrap items-center gap-1.5')}>
          <div className={cn(badge, priority.badge)}>{priority.label}</div>
          <div className={cn(badge, status.badge)}>{status.label}</div>
        </div>

        <div className={cn('flex shrink-0 items-center gap-1')}>
          <button
            type="button"
            className={cn(btnIcon)}
            onClick={() => onEdit(task)}
            aria-label={`Edit ${task.title}`}
          >
            <FiEdit2 />
          </button>
          <button
            type="button"
            className={cn(btnIcon, btnIconDanger)}
            onClick={() => onDelete(task)}
            aria-label={`Delete ${task.title}`}
          >
            <FiTrash2 />
          </button>
        </div>
      </div>

      <div className={cn('mt-3 line-clamp-2 text-base font-bold leading-snug', isCompleted ? 'text-graphite-400 line-through' : 'text-graphite-800')}>
        {task.title}
      </div>
    </article>
  );
};

```

```

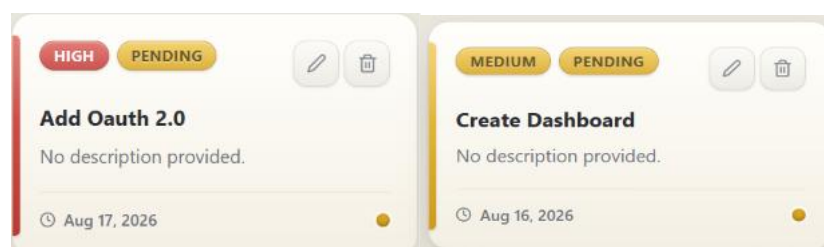
    {/* Description – grows to fill the card so footers align */}
    <p className="mt-1.5 line-clamp-2 flex-1 text-sm leading-relaxed
text-graphite-500">
      {task.description || 'No description provided.'}
    </p>

    {/* Footer: due date + status */}
    <div className="mt-4 flex items-center justify-between border-t
border-paper-200 pt-3">
      <div
        className={cn(
          'flex items-center gap-1.5 text-xs font-medium',
          overdue ? 'text-status-red' : 'text-graphite-500'
        )}
      >
        <FiClock aria-hidden="true" />
        <span>{formatDueDate(task.dueDate)}</span>
        {overdue && (
          <span className="ml-1 rounded-full bg-status-red/10 px-2 py-0.5
text-[10px] font-bold uppercase tracking-wide text-status-red">
            Overdue
          </span>
        )}
      </div>

      <span className={cn(statusDot, status.dot)} title={`Status:
${status.label}`} />
    </div>
  </article>
);
};

export default TaskCard;

```



Dashboard

```
import { useMemo } from 'react';
import { useNavigate } from 'react-router-dom';
import {
  FiActivity,
  FiAlertTriangle,
  FiCheckCircle,
  FiClock,
  FiLayers,
  FiList,
  FiPlus,
} from 'react-icons/fi';
import useTasks from '../hooks/useTasks.js';
import StatsCard from '../components/StatsCard.jsx';
import TaskCard from '../components/TaskCard.jsx';
import Button from '../components/Button.jsx';
import EmptyState from '../components/EmptyState.jsx';
import { cn, paperCard, gloss, emboss, statusDot } from
'../styles/classes.js';
import { formatRelativeDate } from '../utils/format.js';

/** Small paper section header used across the page. */
const SectionHeader = ({ title, caption }) => (
  <div className="flex items-end justify-between gap-3">
    <div>
      <h2 className={cn(emboss, 'font-display text-lg font-bold text-
graphite-700')}>
        {title}
      </h2>
      {caption && <p className="mt-0.5 text-xs text-graphite-
400">{caption}</p>}
    </div>
  </div>
);

/** Quick actions card. */
const QuickActions = ({ onNewTask, onViewAll }) => (
  <div className={cn(paperCard, gloss, 'relative p-5')}>
    <h2 className={cn(emboss, 'font-display text-lg font-bold text-
graphite-700')}>
      Quick Actions
    </h2>
    <div className="mt-4 space-y-2.5">
      <Button variant="primary" size="md" className="w-full"
onClick={onNewTask}>
        <FiPlus />
        New Task
      </Button>
    </div>
  </div>
);
```

```

        <Button
          variant="secondary"
          size="md"
          className="w-full"
          onClick={onViewAll}
        >
        <FiList />
        View All Tasks
      </Button>
    </div>
  </div>
);

/** Recent activity feed derived from task history. */
const ActivityFeed = ({ items }) => (
  <div className={cn(paperCard, gloss, 'relative p-5')}>
    <h2 className={cn(emboss, 'font-display text-lg font-bold text-graphite-700')}>
      Recent Activity
    </h2>
    {items.length === 0 ? (
      <p className="mt-3 text-sm text-graphite-400">No activity yet.</p>
    ) : (
      <ul className="mt-4 space-y-4 border-l-2 border-accent-500/20 pl-4">
        {items.map((item) => (
          <li key={item.id} className="relative">
            <span className={cn(statusDot, 'absolute -left-[21px] top-1.5 bg-status-blue')} />
            <p className="text-sm font-semibold leading-snug text-graphite-700">
              {item.title}
            </p>
            <p className="mt-0.5 text-xs text-graphite-400">
              {item.status === 'completed' ? 'Completed' : 'Added'} · {' '}
              {formatRelativeDate(item.createdAt)}
            </p>
          </li>
        ))}
      </ul>
    )}
  </div>
);

/** Loading skeleton shown while the first fetch is in flight. */
const DashboardSkeleton = () => (
  <div className="space-y-8">
    <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 xl:grid-cols-4">
      {[0, 1, 2, 3].map((n) => (
        <div key={n} className={cn(paperCard, 'h-36 animate-pulse')} />
      ))}
    </div>
    <div className={cn(paperCard, 'h-64 animate-pulse')} />
  </div>
);

```

```
);

/** Dashboard overview page. */
const Dashboard = () => {
  const { tasks, loading, usingFallback } = useTasks();
  const navigate = useNavigate();

  const stats = useMemo(
    () => ({
      total: tasks.length,
      completed: tasks.filter((task) => task.status ===
'completed').length,
      pending: tasks.filter((task) => task.status !== 'completed').length,
      highPriority: tasks.filter((task) => task.priority ===
'high').length,
    }),
    [tasks]
  );

  const recentTasks = useMemo(
    () =>
      [...tasks]
        .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt))
        .slice(0, 4),
    [tasks]
  );

  const activity = useMemo(
    () =>
      [...tasks]
        .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt))
        .slice(0, 6),
    [tasks]
  );

  if (loading) {
    return <DashboardSkeleton />;
  }

  return (
    <div className="space-y-8">
      {/* Offline demo-mode banner */}
      {usingFallback && (
        <div className={cn(paperCard, 'flex items-center gap-3 border-
accent-500/40 px-5 py-4')}>
          <FiActivity className="shrink-0 text-accent-600" aria-
hidden="true" />
          <p className="text-sm text-graphite-600">
            The API is not reachable – showing bundled demo data. Start the
            backend (npm run dev in <code className="font-
mono">backend</code>)
            to use live data.
          </p>
        </div>
      )}
    </div>
  );
}
```

```
    </div>
  })

  { /* Statistics */ }
  <section
    className="grid grid-cols-1 gap-4 sm:grid-cols-2 xl:grid-cols-4"
    aria-label="Statistics"
  >
    <StatsCard
      label="Total Tasks"
      value={stats.total}
      icon={<FiLayers />}
      tone="blue"
      subtitle="Across the workspace"
    />
    <StatsCard
      label="Completed"
      value={stats.completed}
      icon={<FiCheckCircle />}
      tone="green"
      subtitle="Nice work, keep it up"
    />
    <StatsCard
      label="Pending"
      value={stats.pending}
      icon={<FiClock />}
      tone="yellow"
      subtitle="Waiting for action"
    />
    <StatsCard
      label="High Priority"
      value={stats.highPriority}
      icon={<FiAlertTriangle />}
      tone="red"
      subtitle="Needs attention soon"
    />
  </section>

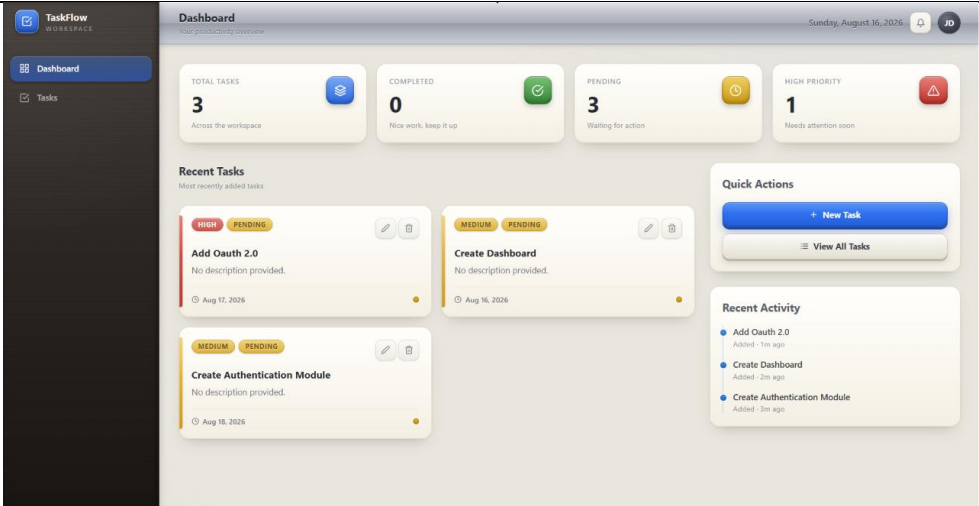
  { /* Content columns */ }
  <section className="grid grid-cols-1 gap-6 lg:grid-cols-3">
    <div className="space-y-6 lg:col-span-2">
      <SectionHeader
        title="Recent Tasks"
        caption="Most recently added tasks"
      />

      {recentTasks.length > 0 ? (
        <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
          {recentTasks.map((task) => (
            <TaskCard
              key={task.id}
              task={task}
              onEdit={(selected) =>
```

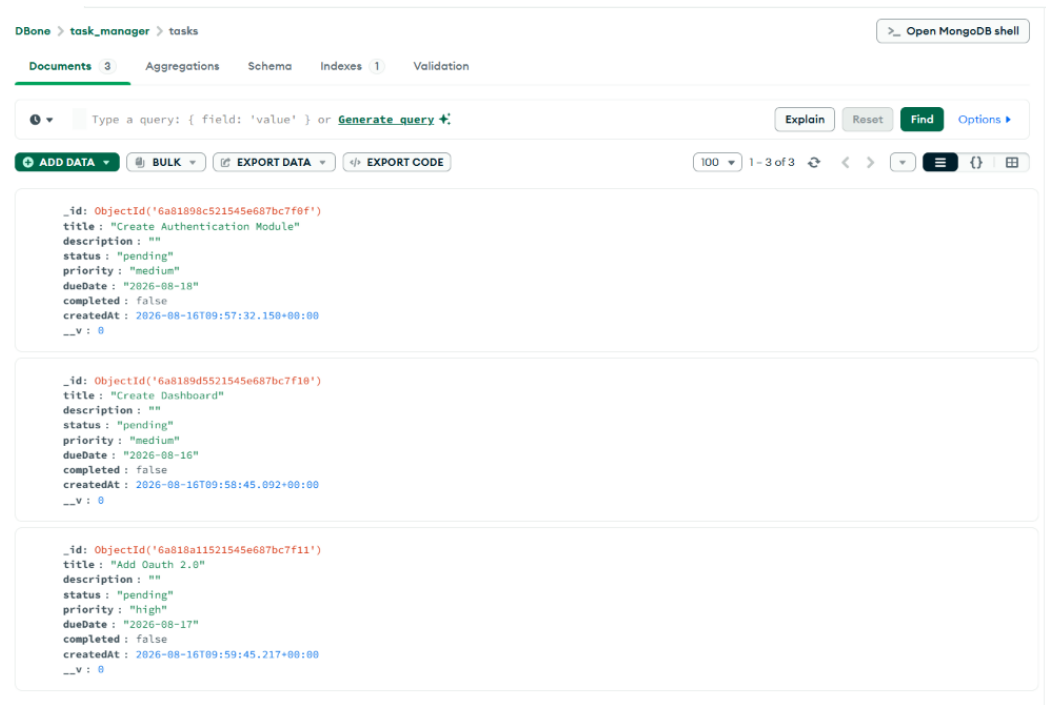
```
        navigate('/tasks', { state: { openModal: selected } })
      }
      onDelete={() =>
        navigate('/tasks', { state: { deleteTask: selected } })
      }
    />
  )))
</div>
) : (
  <EmptyState
    icon={<FiLayers />}
    title="No tasks yet"
    message="Add your first task to see it here on the
dashboard."
    action={
      <Button
        onClick={() => navigate('/tasks', { state: { openModal:
true } })}
      >
        <FiPlus />
        Add Task
      </Button>
    }
  />
)}
</div>

<aside className="space-y-6">
  <QuickActions
    onNewTask={() => navigate('/tasks', { state: { openModal: true
} })}
    onViewAll={() => navigate('/tasks')}
  />
  <ActivityFeed items={activity} />
</aside>
</section>
</div>
);
};

export default Dashboard;
```



Database



Outcome

Students will be able to connect an Express application to MongoDB using Mongoose, define a validated schema, and perform real CRUD operations against a live database.